Goal	Traditional approach	Akka approach (actors)
Make conversation data durable, even if the application restarts or crashes.	Rewrite code into DAOs. Use the database as one big shared mutable state where all parties create, update, insert, and query the data.	Continue to use the in-memory state. Send changes to the state as messages to a log. This log is reread only if the application restarts.
Provide interactive features (mentions).	Poll the database. Polling uses many resources even if there's no change in the data.	Push events to interested parties. The objects notify interested parties only when there's a significant event, reducing overhead.
Decouple services. The mentions and chat features shouldn't interfere with each other.	Add a message queue for asynchronous processing.	No need to add a message queue; actors are asynchronous by definition. No extra complexity; you're familiar with sending and receiving messages.
Prevent failure of the entire system when critical services fail or behave outside of specified performance parameters for any given time.	Try to prevent any error from happening by predicting all failure scenarios and catching exceptions for these scenarios.	Send messages asynchronously. If a message isn't handled by a crashed component, the crash has no effect on the stability of the other components.

It would be great if you could write the application code once and then scale it any way you like. You want to avoid radically changing the application's main objects: for example, the way you had to replace all logic in the in-memory objects with DAOs in section 1.4.1.